

TCS NQT

Complete Coding Prep Guide

Digital Role · Java Edition · March 2026

35

3

Java

Categories

Language

Math Basics — 12 Questions

Strings — 12 Questions

Arrays — 12 Questions

1 Prime Number Check

 $O(\sqrt{n})$

Easy

PROBLEM STATEMENT

Given an integer N , determine if it is prime. A prime number has exactly two factors: 1 and itself. E.g., 7 is prime; 9 is not.

LOGIC

1. If $N \leq 1$, print 'Not Prime' immediately — 0 and 1 are not prime by definition.
2. Loop i from 2 while $i*i \leq N$ (equivalent to $i \leq \sqrt{N}$ but avoids floating-point).
3. If $N \% i == 0$ for any i , a divisor exists — print 'Not Prime' and return.
4. If the loop completes without finding a divisor, print 'Prime'.

JAVA SOLUTION

```
import java.util.Scanner;
public class PrimeCheck {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if (n <= 1) { System.out.println("Not Prime"); return; }
        boolean isPrime = true;
        for (int i = 2; i * i <= n; i++)
            if (n % i == 0) { isPrime = false; break; }
        System.out.println(isPrime ? "Prime" : "Not Prime");
    }
}
```

PROBLEM STATEMENT

Given an integer N, check if it reads the same forwards and backwards. E.g., 121 is a palindrome; 123 is not.

LOGIC

1. Save the original in 'orig' before modifying N.
2. Reverse digits: $rev = rev * 10 + n \% 10$, then $n /= 10$, loop until $n == 0$.
3. If $rev == orig$, it is a palindrome.

JAVA SOLUTION

```
import java.util.Scanner;
public class PalindromeNum {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), orig = n, rev = 0;
        while (n > 0) { rev = rev * 10 + n % 10; n /= 10; }
        System.out.println(orig == rev ? "Palindrome" : "Not Palindrome");
    }
}
```

PROBLEM STATEMENT

Compute $N! = 1 \times 2 \times \dots \times N$. By definition $0! = 1$. Output can be very large — handle overflow carefully.

LOGIC

1. Declare fact = 1 as long (handles 0! correctly and avoids int overflow).
2. Loop i from 2 to N multiplying fact by i each step.
3. Print fact. long holds values up to 20! safely.

JAVA SOLUTION

```
import java.util.Scanner;

public class Factorial {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        long fact = 1;
        for (int i = 2; i <= n; i++) fact *= i;
        System.out.println(fact);
    }
}
```

PROBLEM STATEMENT

Print the first N terms of the Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8 ... Each term equals the sum of the two preceding terms.

LOGIC

1. Initialize a = 0 (first term) and b = 1 (second term).
2. In each iteration, print 'a', compute next = a+b, then shift a=b, b=next.
3. Repeat N times.

JAVA SOLUTION

```
import java.util.Scanner;
public class Fibonacci {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), a = 0, b = 1;
        for (int i = 0; i < n; i++) {
            System.out.print(a + " ");
            int t = a + b; a = b; b = t;
        }
    }
}
```

PROBLEM STATEMENT

A number is Armstrong if it equals the sum of its digits each raised to the power of the total digit count. E.g. $153 = 1^3 + 5^3 + 3^3 = 153$.

LOGIC

1. Count total digits d using `String.valueOf(n).length()`.
2. Extract each digit via `n%10`, raise to power d with `Math.pow()`, cast to int.
3. Accumulate the sum; compare with original.

JAVA SOLUTION

```
import java.util.Scanner;
public class Armstrong {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), orig = n, sum = 0;
        int digits = String.valueOf(n).length();
        while (n > 0) {
            int d = n % 10;
            sum += (int) Math.pow(d, digits);
            n /= 10;
        }
        System.out.println(orig == sum ? "Armstrong" : "Not Armstrong");
    }
}
```

PROBLEM STATEMENT

A number N is perfect if the sum of its proper divisors (all divisors excluding N itself) equals N. E.g., $6 = 1+2+3 = 6$. Print 'Perfect' or 'Not Perfect'.

LOGIC

1. Initialize sum = 1 (1 is always a proper divisor for $N > 1$).
2. Loop i from 2 while $i*i \leq N$; if $N\%i==0$ add i and N/i (avoid double-counting when $i==N/i$).
3. Compare sum with N.

JAVA SOLUTION

```
import java.util.Scanner;
public class PerfectNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if (n <= 1) { System.out.println("Not Perfect"); return; }
        int sum = 1;
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                sum += i;
                if (i != n / i) sum += n / i;
            }
        }
        System.out.println(sum == n ? "Perfect" : "Not Perfect");
    }
}
```

PROBLEM STATEMENT

A Strong number equals the sum of the factorials of its digits. E.g., $145 = 1! + 4! + 5! = 1 + 24 + 120 = 145$.

LOGIC

1. Extract each digit from N using $N \% 10$.
2. Compute the factorial of each extracted digit.
3. Accumulate the sum of factorials.
4. If sum equals original N, it is Strong.

JAVA SOLUTION

```
import java.util.Scanner;
public class StrongNumber {
    static long factorial(int n) {
        long f = 1;
        for (int i = 2; i <= n; i++) f *= i;
        return f;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), orig = n;
        long sum = 0;
        while (n > 0) { sum += factorial(n % 10); n /= 10; }
        System.out.println(sum == orig ? "Strong Number" : "Not Strong Number");
    }
}
```


PROBLEM STATEMENT

Given two integers A and B, compute their Greatest Common Divisor (GCD) and Least Common Multiple (LCM). $LCM = (A * B) / GCD$.

LOGIC

1. Use Euclidean algorithm: $\text{gcd}(a,b) = \text{gcd}(b, a \% b)$ until $b == 0$.
2. Base case: when $b == 0$, return a .
3. $LCM = (a / \text{gcd}) * b$ — divide before multiply to prevent overflow.

JAVA SOLUTION

```
import java.util.Scanner;
public class GcdLcm {
    static int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt(), b = sc.nextInt();
        int g = gcd(a, b);
        long l = (long) a / g * b;
        System.out.println("GCD: " + g + ", LCM: " + l);
    }
}
```

PROBLEM STATEMENT

Given N, find the sum of its digits. Also compute the digital root — repeatedly sum digits until a single digit remains. E.g., $9875 \rightarrow 9+8+7+5 = 29 \rightarrow 2+9 = 11 \rightarrow 1+1 = 2$.

LOGIC

1. Extract each digit with $n\%10$, accumulate into sum, do $n/=10$.
2. For digital root: repeat until $\text{sum} < 10$.
3. Shortcut formula for digital root: $1 + (n-1)\%9$ (works for $n > 0$).

JAVA SOLUTION

```
import java.util.Scanner;
public class SumDigits {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), orig = n, sum = 0;
        while (n > 0) { sum += n % 10; n /= 10; }
        System.out.println("Digit Sum: " + sum);
        // Digital root
        int dr = orig == 0 ? 0 : 1 + (orig - 1) % 9;
        System.out.println("Digital Root: " + dr);
    }
}
```



Print All Prime Factors

$O(\sqrt{n})$

Medium

PROBLEM STATEMENT

Given N , print all its prime factors. E.g., $12 \rightarrow 2\ 2\ 3$ (since $12 = 2 \times 2 \times 3$). Each factor is printed as many times as it divides N .

LOGIC

1. While N is divisible by 2, print 2 and divide N by 2 (handles the only even prime).
2. Loop i from 3 to $\sqrt{N}$ in steps of 2 (skip even numbers).
3. While $N \% i == 0$, print i and divide N by i .
4. If $N > 2$ after the loop, N itself is a prime factor — print it.

JAVA SOLUTION

```
import java.util.Scanner;

public class PrimeFactors {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        while (n % 2 == 0) { System.out.print(2 + " "); n /= 2; }
        for (int i = 3; i * i <= n; i += 2)
            while (n % i == 0) { System.out.print(i + " "); n /= i; }
        if (n > 2) System.out.print(n);
    }
}
```



Power Without Math.pow()

$O(\log b)$

Medium

PROBLEM STATEMENT

Compute A raised to the power B (A^B) without using Math.pow(). Use Fast Exponentiation (binary exponentiation) for efficiency.

LOGIC

1. If exponent $b == 0$, return 1 (anything⁰ = 1).
2. If b is even: compute half = power(a, b/2); return half * half.
3. If b is odd: return a * power(a, b-1).
4. This halves the problem each step — $O(\log b)$ multiplications.

JAVA SOLUTION

```
import java.util.Scanner;
public class FastPower {
    static long power(long a, int b) {
        if (b == 0) return 1;
        if (b % 2 == 0) { long h = power(a, b/2); return h * h; }
        return a * power(a, b - 1);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        long a = sc.nextLong(); int b = sc.nextInt();
        System.out.println(power(a, b));
    }
}
```

1
2

Count and Sum of Even/Odd Numbers (1 to N)

O(1)

Easy

PROBLEM STATEMENT

Given N, count how many even and odd numbers exist from 1 to N, and compute their separate sums without looping.

LOGIC

1. Even count = $N/2$. Odd count = $N - N/2$.
2. Sum of even numbers 1..N: let $k = N/2$; $\text{sum_even} = k*(k+1)$.
3. Sum of odd numbers: $\text{sum_odd} = \text{total_sum} - \text{sum_even}$ where $\text{total} = N*(N+1)/2$.

JAVA SOLUTION

```
import java.util.Scanner;
public class EvenOddCount {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        long n = sc.nextLong();
        long evenCount = n / 2, oddCount = n - evenCount;
        long totalSum = n * (n + 1) / 2;
        long k = n / 2;
        long sumEven = k * (k + 1);
        long sumOdd = totalSum - sumEven;
        System.out.println("Even count: " + evenCount + ", Sum: " + sumEven);
        System.out.println("Odd count: " + oddCount + ", Sum: " + sumOdd);
    }
}
```

■ PRO TIPS — Avoid These TCS Compiler Pitfalls

- Use 'long' for factorial/power results — int overflows silently at 13!.
- Write $i*i \leq n$ not $i \leq \text{Math.sqrt}(n)$ — avoids floating-point edge-case bugs.
- After `sc.nextInt()`, add `sc.nextLine()` to flush the buffer before reading a string.
- Cast `Math.pow(d, digits)` to (int) for Armstrong — double comparison fails silently.
- Always test edge cases: $N=0$, $N=1$, $N=2$ for primes, and $N=0$ for factorials.



STRINGS

Anagram, Palindrome, Cipher, Pangram, Compression & more

1 Anagram Check

$O(n \log n)$

Easy

PROBLEM STATEMENT

Two strings are anagrams if they contain the same characters with the same frequencies in any order. E.g. 'listen' and 'silent'. Print 'Anagram' or 'Not Anagram'.

LOGIC

1. If lengths differ, print 'Not Anagram' immediately — no need to sort.
2. Convert both to lowercase for case-insensitive comparison.
3. Convert to char arrays and sort both with `Arrays.sort()`.
4. `Arrays.equals()` checks if sorted arrays are identical.

JAVA SOLUTION

```
import java.util.Arrays; import java.util.Scanner;
public class Anagram {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        char[] a = sc.next().toLowerCase().toCharArray();
        char[] b = sc.next().toLowerCase().toCharArray();
        if (a.length != b.length) { System.out.println("Not Anagram"); return; }
        Arrays.sort(a); Arrays.sort(b);
        System.out.println(Arrays.equals(a, b) ? "Anagram" : "Not Anagram");
    }
}
```

PROBLEM STATEMENT

Count vowels (a, e, i, o, u) and consonants in a string. Ignore spaces, digits, and special characters.

LOGIC

1. Convert to lowercase for uniform comparison.
2. For each char: if it is in 'aeiou', increment vowel count.
3. Else if `Character.isLetter()` is true, increment consonant count.

JAVA SOLUTION

```
import java.util.Scanner;

public class VowelConsonant {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine().toLowerCase();
        int v = 0, c = 0;
        for (char ch : s.toCharArray()) {
            if ("aeiou".indexOf(ch) != -1) v++;
            else if (Character.isLetter(ch)) c++;
        }
        System.out.println("Vowels: " + v + ", Consonants: " + c);
    }
}
```

3

Reverse a String

 $O(n)$

Easy

PROBLEM STATEMENT

Given a string, print its reverse. E.g., 'hello world' → 'dlrow olleh'. The entire input including spaces must be reversed.

LOGIC

1. Use StringBuilder which provides a built-in reverse() method.
2. Wrap the original string in new StringBuilder(s).
3. Call .reverse().toString() and print.

JAVA SOLUTION

```
import java.util.Scanner;
public class ReverseStr {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        System.out.println(new StringBuilder(s).reverse());
    }
}
```


PROBLEM STATEMENT

Compress a string using run-length encoding: replace consecutive repeating characters with the character followed by its count. Omit count if it is 1. E.g., 'aaabbc' → 'a3b2c'.

LOGIC

1. Use pointer i=0 to traverse the string character by character.
2. For each char, count how many times it repeats consecutively.
3. Append the char to StringBuilder; append count only if count > 1.
4. Advance i by count and continue.

JAVA SOLUTION

```
import java.util.Scanner;

public class StringCompression {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.next();
        StringBuilder sb = new StringBuilder();
        int i = 0;
        while (i < s.length()) {
            char c = s.charAt(i); int count = 0;
            while (i < s.length() && s.charAt(i) == c) { i++; count++; }
            sb.append(c); if (count > 1) sb.append(count);
        }
        System.out.println(sb);
    }
}
```

PROBLEM STATEMENT

Check if a string reads the same forwards and backwards. E.g., 'racecar' is a palindrome; 'hello' is not. Print 'Palindrome' or 'Not Palindrome'.

LOGIC

1. Convert to lowercase for case-insensitive comparison.
2. Reverse using `new StringBuilder(s).reverse().toString()`.
3. Compare with `.equals()` — never use `==` for strings.

JAVA SOLUTION

```
import java.util.Scanner;
public class PalindromeStr {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine().toLowerCase();
        String rev = new StringBuilder(s).reverse().toString();
        System.out.println(s.equals(rev) ? "Palindrome" : "Not Palindrome");
    }
}
```

PROBLEM STATEMENT

Given a string and a target character, count how many times the character appears in the string (case-insensitive). E.g., 'programming', 'g' → 2.

LOGIC

1. Convert both the string and target char to lowercase.
2. Traverse each character in the string.
3. If it equals the target, increment counter.

JAVA SOLUTION

```
import java.util.Scanner;
public class CharCount {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine().toLowerCase();
        char target = sc.next().toLowerCase().charAt(0);
        int count = 0;
        for (char c : s.toCharArray())
            if (c == target) count++;
        System.out.println("Count: " + count);
    }
}
```

PROBLEM STATEMENT

Remove all duplicate characters from a string keeping only the first occurrence, preserving original order.
E.g., 'programming' → 'progamin'.

LOGIC

1. Use a LinkedHashSet to store seen characters (preserves insertion order).
2. Iterate over the string; add each character to the set.
3. Build the result by iterating the set and appending each character.

JAVA SOLUTION

```
import java.util.LinkedHashSet; import java.util.Scanner;
public class RemoveDupChars {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        LinkedHashSet<Character> seen = new LinkedHashSet<>();
        for (char c : s.toCharArray()) seen.add(c);
        StringBuilder sb = new StringBuilder();
        for (char c : seen) sb.append(c);
        System.out.println(sb);
    }
}
```

PROBLEM STATEMENT

A pangram contains every letter of the alphabet at least once. E.g., 'The quick brown fox jumps over the lazy dog' is a pangram. Print 'Pangram' or 'Not Pangram'.

LOGIC

1. Convert the string to lowercase.
2. Add all characters to a HashSet.
3. Check if the set contains all 26 letters a through z.

JAVA SOLUTION

```
import java.util.HashSet; import java.util.Scanner;
public class Pangram {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine().toLowerCase();
        HashSet<Character> set = new HashSet<>();
        for (char c : s.toCharArray())
            if (c >= 'a' && c <= 'z') set.add(c);
        System.out.println(set.size() == 26 ? "Pangram" : "Not Pangram");
    }
}
```

PROBLEM STATEMENT

Encrypt a string using Caesar Cipher — shift each letter forward by K positions (wrapping around z → a). Non-letter characters are unchanged.

LOGIC

1. For each character, check if it is a letter.
2. Determine the base: 'a' for lowercase, 'A' for uppercase.
3. New char = base + (ch - base + k) % 26 — the modulo handles wrap-around.

JAVA SOLUTION

```
import java.util.Scanner;
public class CaesarCipher {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine(); int k = sc.nextInt() % 26;
        StringBuilder sb = new StringBuilder();
        for (char c : s.toCharArray()) {
            if (Character.isLetter(c)) {
                char base = Character.isUpperCase(c) ? 'A' : 'a';
                sb.append((char)(base + (c - base + k) % 26));
            } else sb.append(c);
        }
        System.out.println(sb);
    }
}
```

1
0

Most Frequent Character

 $O(n)$

Easy

PROBLEM STATEMENT

Find the character that appears most frequently in a string. If there is a tie, return the character that appears first. E.g., 'abcabd' → 'a' (appears 2 times).

LOGIC

1. Use an `int[256]` frequency array (covers all ASCII characters).
2. Traverse the string and increment `freq[ch]` for each character.
3. Find the index with the maximum value in the frequency array.

JAVA SOLUTION

```
import java.util.Scanner;
public class MostFreqChar {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int[] freq = new int[256];
        for (char c : s.toCharArray()) freq[c]++;
        char maxChar = s.charAt(0); int maxFreq = 0;
        for (char c : s.toCharArray())
            if (freq[c] > maxFreq) { maxFreq = freq[c]; maxChar = c; }
        System.out.println("'" + maxChar + "' appears " + maxFreq + " times");
    }
}
```



Toggle Case of String

$O(n)$

Easy

PROBLEM STATEMENT

Toggle the case of each character in a string — uppercase letters become lowercase and vice versa. E.g., 'Hello World' → 'hELLO wORLD'.

LOGIC

1. Traverse each character in the string.
2. Use `Character.isUpperCase()` and `Character.isLowerCase()` for detection.
3. Apply `Character.toLowerCase()` or `toUpperCase()` to toggle accordingly.

JAVA SOLUTION

```
import java.util.Scanner;
public class ToggleCase {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        StringBuilder sb = new StringBuilder();
        for (char c : s.toCharArray()) {
            if (Character.isUpperCase(c)) sb.append(Character.toLowerCase(c));
            else if (Character.isLowerCase(c)) sb.append(Character.toUpperCase(c));
            else sb.append(c);
        }
        System.out.println(sb);
    }
}
```


1
2

Check String Rotation

 $O(n)$

Medium

PROBLEM STATEMENT

Given two strings A and B of the same length, determine if B is a rotation of A. E.g., 'abcde' and 'cdeab' — B is A rotated by 2 positions.

LOGIC

1. If lengths differ, return false immediately.
2. Concatenate A with itself: doubled = A + A.
3. If doubled.contains(B), then B is a rotation of A.

JAVA SOLUTION

```
import java.util.Scanner;
public class StringRotation {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String a = sc.next(), b = sc.next();
        if (a.length() != b.length()) {
            System.out.println("Not a rotation"); return;
        }
        System.out.println((a + a).contains(b) ? "Rotation" : "Not a rotation");
    }
}
```

■ PRO TIPS — Avoid These TCS Compiler Pitfalls

- ALWAYS use .equals() not == for string comparison — == compares references, not content.
- Use sc.nextLine() for strings with spaces — sc.next() silently truncates at whitespace.
- After sc.nextInt() followed by sc.nextLine(), add an extra sc.nextLine() to flush the newline.
- For anagram: check length first — if unequal, skip sorting (free early exit).
- StringBuilder is far faster than String += in loops — String concatenation creates new objects.

1 Find Min and Max Element $O(n)$

Easy

PROBLEM STATEMENT

Given an array of N integers, find both the minimum and maximum values in a single traversal without sorting.

LOGIC

1. Initialize min = Integer.MAX_VALUE and max = Integer.MIN_VALUE.
2. Traverse all elements; update min if element is smaller, max if larger.
3. Print both after the single loop.

JAVA SOLUTION

```
import java.util.Scanner;
public class MinMax {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int min = Integer.MAX_VALUE, max = Integer.MIN_VALUE;
        for (int i = 0; i < n; i++) {
            int x = sc.nextInt();
            if (x < min) min = x;
            if (x > max) max = x;
        }
        System.out.println("Min: " + min + ", Max: " + max);
    }
}
```

PROBLEM STATEMENT

Sort an array in ascending order using Bubble Sort — implement manually, not with `Arrays.sort()`. TCS expects you to demonstrate understanding of the algorithm.

LOGIC

1. Outer loop runs $N-1$ passes; each pass bubbles the largest unsorted element to its position.
2. Inner loop runs from 0 to $n-i-2$; compare adjacent pairs and swap if out of order.
3. Use a temp variable for swapping.

JAVA SOLUTION

```
import java.util.Scanner;
public class BubbleSort {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); int[] a = new int[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();
        for (int i = 0; i < n - 1; i++)
            for (int j = 0; j < n - i - 1; j++)
                if (a[j] > a[j+1]) { int t=a[j]; a[j]=a[j+1]; a[j+1]=t; }
        for (int x : a) System.out.print(x + " ");
    }
}
```

PROBLEM STATEMENT

Search for a key value in an unsorted array and return its first index. Print the index if found, otherwise print 'Not Found'.

LOGIC

1. Traverse from index 0 to N-1.
2. If $a[i] == \text{key}$, store index and break.
3. Print index or 'Not Found' based on whether a match was found.

JAVA SOLUTION

```
import java.util.Scanner;
public class LinearSearch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); int[] a = new int[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();
        int key = sc.nextInt(), idx = -1;
        for (int i = 0; i < n; i++) if (a[i] == key) { idx = i; break; }
        System.out.println(idx == -1 ? "Not Found" : "Found at index " + idx);
    }
}
```

PROBLEM STATEMENT

Remove all duplicate integers from an array and print only the unique elements while preserving their original insertion order.

LOGIC

1. Use a LinkedHashSet — it stores unique values in insertion order.
2. Add all elements; duplicates are silently rejected.
3. Iterate and print the set.

JAVA SOLUTION

```
import java.util.LinkedHashSet; import java.util.Scanner;
public class RemoveDups {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        LinkedHashSet<Integer> set = new LinkedHashSet<>();
        for (int i = 0; i < n; i++) set.add(sc.nextInt());
        for (int x : set) System.out.print(x + " ");
    }
}
```

PROBLEM STATEMENT

Find the second largest distinct element in an array. Handle edge cases like all-identical elements. E.g., [3,1,4,1,5,9,2,6] → 6.

LOGIC

1. Track first and second, both = Integer.MIN_VALUE.
2. If $x > \text{first}$: $\text{second} = \text{first}$, $\text{first} = x$.
3. Else if $x > \text{second}$ AND $x \neq \text{first}$: $\text{second} = x$. (The $\neq$ first guard is critical!)

JAVA SOLUTION

```
import java.util.Scanner;
public class SecondLargest {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
        for (int i = 0; i < n; i++) {
            int x = sc.nextInt();
            if (x > first) { second = first; first = x; }
            else if (x > second && x != first) second = x;
        }
        if (second == Integer.MIN_VALUE) System.out.println("No second largest");
        else System.out.println("Second Largest: " + second);
    }
}
```

PROBLEM STATEMENT

Compute the sum and average of all elements in an array of N integers. Format average to 2 decimal places.

LOGIC

1. Declare sum as long to handle large totals without overflow.
2. Traverse the array once, accumulating sum.
3. Compute average = (double) sum / n and use `String.format("%.2f")` for 2 decimal places.

JAVA SOLUTION

```
import java.util.Scanner;
public class SumAverage {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); long sum = 0;
        for (int i = 0; i < n; i++) sum += sc.nextInt();
        System.out.println("Sum: " + sum);
        System.out.printf("Average: %.2f\n", (double) sum / n);
    }
}
```

PROBLEM STATEMENT

Reverse the elements of an array in-place without using any extra array. E.g., [1, 2, 3, 4, 5] → [5, 4, 3, 2, 1]

LOGIC

1. Use two pointers: left = 0 and right = n-1.
2. While left < right, swap a[left] and a[right].
3. Increment left and decrement right each iteration.

JAVA SOLUTION

```
import java.util.Scanner;

public class ReverseArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); int[] a = new int[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();
        int l = 0, r = n - 1;
        while (l < r) { int t = a[l]; a[l++] = a[r]; a[r--] = t; }
        for (int x : a) System.out.print(x + " ");
    }
}
```


PROBLEM STATEMENT

Search for a key in a sorted array using Binary Search. Return the index if found, otherwise -1. The array must be sorted before applying binary search.

LOGIC

1. Set low = 0, high = n-1.
2. Compute mid = low + (high - low) / 2 (avoids integer overflow vs (low+high)/2).
3. If $a[mid] == key$, return mid. If $key < a[mid]$, high = mid-1. Else low = mid+1.
4. If low > high, key is not present — return -1.

JAVA SOLUTION

```
import java.util.Scanner;
public class BinarySearch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); int[] a = new int[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();
        int key = sc.nextInt(), low = 0, high = n - 1, res = -1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (a[mid] == key) { res = mid; break; }
            else if (key < a[mid]) high = mid - 1;
            else low = mid + 1;
        }
        System.out.println(res == -1 ? "Not Found" : "Found at index " + res);
    }
}
```

PROBLEM STATEMENT

Print the frequency of each distinct element in an array. Elements must be printed in the order they first appear, with no element printed twice.

LOGIC

1. Use a LinkedHashMap to store element → frequency.
2. For each element, use `getOrDefault(x, 0) + 1` to increment.
3. Iterate the map and print each entry.

JAVA SOLUTION

```
import java.util.*;
public class FreqCount {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        LinkedHashMap<Integer, Integer> map = new LinkedHashMap<>();
        for (int i = 0; i < n; i++) {
            int x = sc.nextInt();
            map.put(x, map.getOrDefault(x, 0) + 1);
        }
        for (Map.Entry<Integer, Integer> e : map.entrySet())
            System.out.println(e.getKey() + " -> " + e.getValue());
        }
    }
```

10

Rotate Array by K Positions

 $O(n)$

Medium

PROBLEM STATEMENT

Rotate an array to the right by K positions. E.g., [1,2,3,4,5] rotated right by 2 $\rightarrow$ [4,5,1,2,3]. Handle $K > N$ by using $K \% N$.

LOGIC

1. Normalize: $k = k \% n$ (handles $k > n$).
2. Reverse the entire array.
3. Reverse first k elements, then reverse remaining $n-k$ elements.
4. This three-reversal trick achieves rotation in $O(n)$ with $O(1)$ space.

JAVA SOLUTION

```
import java.util.Scanner;
public class RotateArray {
    static void reverse(int[] a, int l, int r) {
        while (l < r) { int t=a[l]; a[l++]=a[r]; a[r--]=t; }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); int[] a = new int[n];
        for (int i=0; i<n; i++) a[i]=sc.nextInt();
        int k = sc.nextInt() % n;
        reverse(a, 0, n-1); reverse(a, 0, k-1); reverse(a, k, n-1);
        for (int x : a) System.out.print(x + " ");
    }
}
```



Find Missing Number (1 to N)

$O(n)$

Easy

PROBLEM STATEMENT

An array contains $N-1$ distinct integers from 1 to N . One number is missing. Find it in $O(n)$ time and $O(1)$ space without sorting.

LOGIC

1. Compute expected sum = $N*(N+1)/2$ using the arithmetic series formula.
2. Compute actual sum by adding all array elements.
3. Missing number = expected sum - actual sum.

JAVA SOLUTION

```
import java.util.Scanner;
public class MissingNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        long expected = (long) n * (n + 1) / 2, actual = 0;
        for (int i = 0; i < n - 1; i++) actual += sc.nextInt();
        System.out.println("Missing: " + (expected - actual));
    }
}
```

1
2

Selection Sort

 $O(n^2)$

Easy

PROBLEM STATEMENT

Sort an array using Selection Sort — repeatedly find the minimum element in the unsorted portion and place it at the beginning. Implement manually.

LOGIC

1. Outer loop i from 0 to $n-2$; assume $\text{minIdx} = i$.
2. Inner loop j from $i+1$ to $n-1$; update minIdx whenever $a[j] < a[\text{minIdx}]$.
3. Swap $a[i]$ with $a[\text{minIdx}]$ after the inner loop.

JAVA SOLUTION

```
import java.util.Scanner;
public class SelectionSort {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); int[] a = new int[n];
        for (int i=0; i<n; i++) a[i]=sc.nextInt();
        for (int i=0; i<n-1; i++) {
            int minIdx = i;
            for (int j=i+1; j<n; j++) if (a[j]<a[minIdx]) minIdx=j;
            int t=a[i]; a[i]=a[minIdx]; a[minIdx]=t;
        }
        for (int x : a) System.out.print(x + " ");
    }
}
```

■ PRO TIPS — Avoid These TCS Compiler Pitfalls

- Never initialize $\text{min}=0$ or $\text{max}=0$ — use `Integer.MAX_VALUE` / `MIN_VALUE` to handle negative arrays.
- Use $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$ in binary search — $(\text{low}+\text{high})/2$ can overflow for large inputs.
- The $x \neq \text{first}$ guard in second-largest is not optional — skip it and `[5,5,5]` gives wrong answer.
- Use `LinkedHashSet` not `HashSet` when the problem says preserve insertion order.
- Use `long` for sum of array elements — `int` overflows when summing large arrays.



QUICK REFERENCE

Complexity cheat sheet & exam strategy

Topic	Algorithm	Time	Space	Key Trick
Prime Check	Trial division	$O(\sqrt{n})$	$O(1)$	$i*i \leq n$ loop
Factorial	Iterative	$O(n)$	$O(1)$	Use long
GCD	Euclidean	$O(\log n)$	$O(1)$	$\text{gcd}(b, a\%b)$
Fibonacci	Iterative	$O(n)$	$O(1)$	Two variables
Anagram	Sort + compare	$O(n \log n)$	$O(n)$	Length check first
Palindrome str	StringBuilder.reverse	$O(n)$	$O(n)$	.equals() not ==
Caesar Cipher	Char arithmetic	$O(n)$	$O(1)$	$(c - \text{base} + k) \% 26$
Bubble Sort	Comparison swap	$O(n^2)$	$O(1)$	$n-i-1$ inner limit
Binary Search	Divide & conquer	$O(\log n)$	$O(1)$	$\text{low} + (\text{high} - \text{low}) / 2$
Missing Number	Sum formula	$O(n)$	$O(1)$	$n*(n+1)/2 - \text{sum}$
Rotate Array	Triple reverse	$O(n)$	$O(1)$	$k = k \% n$
Remove Dups	LinkedHashSet	$O(n)$	$O(n)$	Preserves order
Frequency	LinkedHashMap	$O(n)$	$O(n)$	getOrDefault
Second Largest	Two variables	$O(n)$	$O(1)$	$x \neq \text{first}$ guard



5-DAY BATTLE PLAN

Mar 16 → Mar 20 — Digital Role

Day 1 — Mar 16

Code Q1-Q6 from memory. Focus: Prime, Factorial, Fibonacci, Armstrong, Perfect, Strong.

Day 2 — Mar 17

Code Q7-Q12: GCD/LCM, Sum of Digits, Prime Factors, Fast Power, Even/Odd formulas.

Day 3 — Mar 18

Code all 12 string problems. Focus extra time on Compression, Caesar Cipher, Rotation.

Day 4 — Mar 19

Code all 12 array problems. Priority: Binary Search, Rotate Array, Missing Number.

Day 5 — Mar 20

Only review Pro Tips sections. Fix the `sc.nextLine()` buffer bug habit. Stay calm — you have this.

All the best for TCS NQT — Digital Role · March 20, 2026